Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №14

Дисциплина: Основы программирования на Python

Тема: Разработка REST API на Python с использованием FastAPI

Вариант 2: Библиотека — книги

Цель работы

Получить практические навыки разработки современных REST API на Python с использованием фреймворка FastAPI. В ходе работы необходимо познакомиться с FastAPI и структурой backend-приложения, научиться создавать HTTP endpoints, использовать методы GET, POST, PUT, PATCH и DELETE, работать с параметрами URL и query-параметрами, принимать и возвращать JSON, применять Pydantic для валидации данных, использовать коды состояния HTTP, организовать обработку ошибок, получить автоматическую документацию OpenAPI, разделить приложение на слои, подключить базу данных, использовать механизм Dependency Injection, реализовать пагинацию и фильтрацию, а также написать автоматические тесты API.

Постановка индивидуального задания (Вариант 2. Библиотека — книги)

Разработать REST API для сущности «книга» с базовым путём /books. Книга описывается полями: название, автор, жанр, год издания, ISBN, издательство, количество экземпляров и рейтинг. API должно поддерживать создание книги, получение списка с фильтрацией, сортировкой и пагинацией, получение книги по идентификатору, полное и частичное изменение, удаление, поиск по строке и получение статистики: количество книг, средний и максимальный рейтинг, суммарное число экземпляров и распределение по жанрам.

Краткие теоретические сведения

FastAPI — современный асинхронный веб-фреймворк, построенный на Starlette и Pydantic. Маршруты объявляются декораторами router.get, router.post, router.put, router.patch и router.delete; параметры пути указываются в шаблоне адреса в фигурных скобках, а query-параметры объявляются аргументами функции с типом и значением по умолчанию. Фреймворк автоматически преобразует типы и возвращает ошибку 422 при неправильных данных.

Pydantic описывает схемы данных в виде классов с аннотациями и ограничениями Field. Схемы разделяются по назначению: BookCreate задаёт обязательные поля и правила проверки, BookUpdate содержит все поля необязательными для частичного изменения, BookResponse определяет формат ответа. Валидаторы на уровне модели, объявленные декоратором model_validator, проверяют правила, зависящие от нескольких полей, например соотношение доступных и общих экземпляров.

Коды состояния сообщают результат операции: 200 — успешное чтение, 201 — ресурс создан, 204 — успешно без тела ответа, 404 — ресурс не найден, 409 — конфликт данных, 422 — ошибка валидации. OpenAPI — спецификация описания API, которую FastAPI формирует автоматически по аннотациям типов, схемам Pydantic и параметрам декораторов; на её основе строятся интерактивные страницы документации /docs и /redoc.

Dependency Injection в FastAPI реализуется функцией Depends: зависимости объявляются в сигнатуре обработчика и создаются фреймворком, что позволяет подменять их в тестах. Асинхронные обработчики выполняются в event loop, а синхронные и блокирующие операции выносятся в пул потоков функцией run_in_threadpool, чтобы не блокировать цикл событий.

Архитектура приложения

Приложение разделено на слои. Обработчики маршрутов принимают запрос, валидируют данные схемами Pydantic и вызывают сервис. Сервис

содержит бизнес-правила, проверку конфликтов и работу с кэшем. Репозиторий выполняет запросы к базе данных через SQLAlchemy. Зависимости создаются функциями модуля deps.py и внедряются в обработчики через Depends.

Файл	Назначение
app/main.py	Создание приложения, middleware журналирования, обработчик ошибок, маршрут health
app/routers/books.py	Маршруты /books с описанием, схемами ответа и кодами состояния
app/schemas/book.py	Схемы Pydantic: BookCreate, BookUpdate, BookResponse, BookListResponse, BookStatistics
app/services/book.py	BookService: списки, чтение, создание, замена, частичное изменение, удаление, поиск, статистика
app/repositories/book.py	BookRepository: запросы SQLAlchemy, фильтрация, сортировка, пагинация, статистика
app/models/book.py	ORM-модель Book с ограничениями целостности
app/cache.py, app/deps.py	Кэш книг на Redis с резервной реализацией в памяти, фабрики зависимостей
app/errors.py	Исключения AppError, NotFoundError, ConflictError, ValidationError
tests/test_api.py	Двадцать два теста API на тестовом клиенте

Схемы данных и валидация

Схема BookCreate задаёт ограничения для каждого поля: длина названия, автора, жанра и издательства, диапазон года от 1400 до 2100, неотрицательное количество экземпляров, рейтинг от нуля до пяти и ISBN из десяти или тринадцати цифр, проверяемый регулярным выражением. Валидатор модели проверяет, что число доступных экземпляров не превышает общего, и по умолчанию приравнивает доступные к общим. Схема BookUpdate повторяет те же ограничения, но все поля объявлены необязательными, что позволяет изменять отдельные атрибуты.

Листинг 14.1 — Схемы Pydantic (app/schemas/book.py)

```
ISBN_PATTERN = r"^(?:d{10}|d{13})$"


class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    author: str = Field(min_length=1, max_length=120)
    genre: str = Field(min_length=1, max_length=60)
    year: int = Field(ge=1400, le=2100)
    isbn: str = Field(pattern=ISBN_PATTERN)
    publisher: str = Field(min_length=1, max_length=120)
    copies_total: int = Field(ge=0)
```

```
    copies_available: Optional[int] = Field(default=None, ge=0)
    rating: float = Field(ge=0, le=5)


    @model_validator(mode="after")
    def check_copies(self) -> "BookCreate":
        if self.copies_available is None:
            object.__setattr__(self, "copies_available",
                               self.copies_total)
        if self.copies_available > self.copies_total:
            raise ValueError("copies_available cannot exceed copies_total")
        return self
```

Реализованные endpoints

Метод и путь	Параметры	Результат
GET /books	genre, author, min_rating, sort, order, limit, offset	200, объект с полями total, limit, offset и items
GET /books/search	q (обязательный), limit	200, массив найденных книг
GET /books/statistics	нет	200, количество книг, средний и максимальный рейтинг, экземпляры, распределение по жанрам
GET /books/{id}	идентификатор в пути	200 — книга, 404 — не найдена, 422 — неверный тип
POST /books	тело BookCreate	201 — созданная книга, 409 — ISBN уже существует, 422 — ошибка валидации
PUT /books/{id}	тело BookCreate	200 — заменённая книга, 404, 409
PATCH /books/{id}	тело BookUpdate	200 — изменённая книга, 404
DELETE /books/{id}	идентификатор в пути	204 без тела, 404

Список книг возвращается объектом пагинации: total содержит общее число подходящих записей, limit и offset — параметры страницы, items — массив книг. Такая структура позволяет клиенту строить постраничный вывод, не выполняя дополнительный запрос за общим количеством. Параметры сортировки ограничены регулярным выражением, поэтому в запрос нельзя передать произвольное имя поля.

Обработка ошибок и журналирование

Ошибки предметной области описаны иерархией с базовым классом AppError, от которого наследуются NotFoundError, ConflictError и ValidationError. В приложении зарегистрирован единый обработчик, который

преобразует исключение в JSON-ответ с соответствующим кодом состояния: 404 для отсутствующей книги, 409 для конфликта ISBN, 400 для нарушения правил. Ошибки валидации Pydantic обрабатываются самим FastAPI и возвращаются с кодом 422 и указанием поля, в котором обнаружена проблема.

Middleware журналирования фиксирует метод, путь, код ответа и время обработки каждого запроса, что упрощает диагностику. Маршрут /health проверяет доступность приложения и базы данных и используется при развёртывании.

Листинг 14.2 — Обработчики маршрутов (app/routers/books.py)

```
@router.get("", response_model=BookListResponse, summary="Список книг")
async def list_books(
    genre: Optional[str] = Query(None, description="Фильтр по жанру"),
    author: Optional[str] = Query(None),
    min_rating: Optional[float] = Query(None, ge=0, le=5),
    sort: str = Query("title", pattern="^(title|author|year|rating)$"),
    order: str = Query("asc", pattern="^(asc|desc)$"),
    limit: int = Query(20, ge=1, le=100),
    offset: int = Query(0, ge=0),
    service: BookService = Depends(get_service),
):
    items, total = await run_in_threadpool(
        partial(service.list_books, genre=genre, author=author,
                min_rating=min_rating, sort=sort, order=order,
                limit=limit, offset=offset))
    return {"total": total, "limit": limit, "offset": offset,
            "items": items}


@router.post("", response_model=BookResponse,
             status_code=status.HTTP_201_CREATED,
             summary="Создание книги")
def create_book(payload: BookCreate,
                service: BookService = Depends(get_service)):
    return service.create_book(payload.model_dump())
```

Кэширование и зависимости

Чтение книги по идентификатору кэшируется по стратегии Cache-Aside: сервис сначала обращается к кэшу, при промахе читает запись из базы и сохраняет её. Любая операция изменения или удаления инвалидирует соответствующий ключ. Реализация кэша имеет два режима: работа с Redis при его доступности и хранение в памяти процесса, что позволяет запускать приложение и тесты без внешнего сервера.

Зависимости объявлены функциями get_db(), get_cache() и get_service(). Обработчик получает сервис через Depends, а сервис — сессию базы и кэш через конструктор. В тестах зависимость get_db подменяется на сессию временной базы, что позволяет проверять API на изолированных данных.

Документация OpenAPI

Благодаря аннотациям типов, схемам Pydantic и параметрам декораторов фреймворк автоматически формирует спецификацию OpenAPI. Интерактивная документация доступна по адресу /docs, альтернативная — по адресу /redoc. Каждый маршрут снабжён кратким описанием summary и пояснением description, а возможные ошибки перечислены в параметре responses, поэтому документация содержит не только структуру запросов и ответов, но и перечень кодов состояния. Тест openapi_schema проверяет, что спецификация содержит все объявленные пути.

Раздел тестирования

API проверялось двадцатью двумя автоматическими тестами на тестовом клиенте, который обращается к приложению напрямую, без запуска сетевого сервера. Каждый тест работает с временной базой данных SQLite.

№	Тест	Проверяемое поведение
1	test_health	Маршрут /health возвращает 200 и статус ok
2	test_list_books	Список книг возвращает структуру с total и items
3-4	test_get_existing_book, test_get_missing_book	Чтение существующей книги и ответ 404 для отсутствующей
5	test_invalid_path_parameter	Строка вместо идентификатора даёт 422
6-8	test_create_book, test_create_invalid_book, test_create_duplicate_isbn	Создание с кодом 201, ошибка валидации 422, конфликт ISBN 409
9-10	test_replace_book, test_replace_missing_book	Полная замена книги и ответ 404
11-12	test_patch_book, test_patch_validation_error	Частичное изменение и ошибка валидации
13-14	test_delete_book_then_not_found, test_delete_twice	Удаление с кодом 204 и повторное удаление с 404
15-16	test_filter_by_genre, test_filter_by_min_rating	Фильтрация по жанру и минимальному рейтингу

№	Тест	Проверяемое поведение
17-18	test_sorting, test_pagination	Сортировка по полю и направлению, постраничная выдача
19-20	test_search, test_statistics	Поиск по строке и статистика фонда
21	test_cache_aside_and_invalidation	Повторное чтение берётся из кэша, изменение инвалидирует ключ
22	test_openapi_schema	Спецификация OpenAPI содержит объявленные пути

Все тесты проходят успешно. Дополнительно API проверялось вручную через интерактивную документацию /docs: создавалась книга, выполнялся поиск, проверялись фильтры и сортировка, изменялся рейтинг методом PATCH, затем книга удалялась, а повторное обращение возвращало 404.

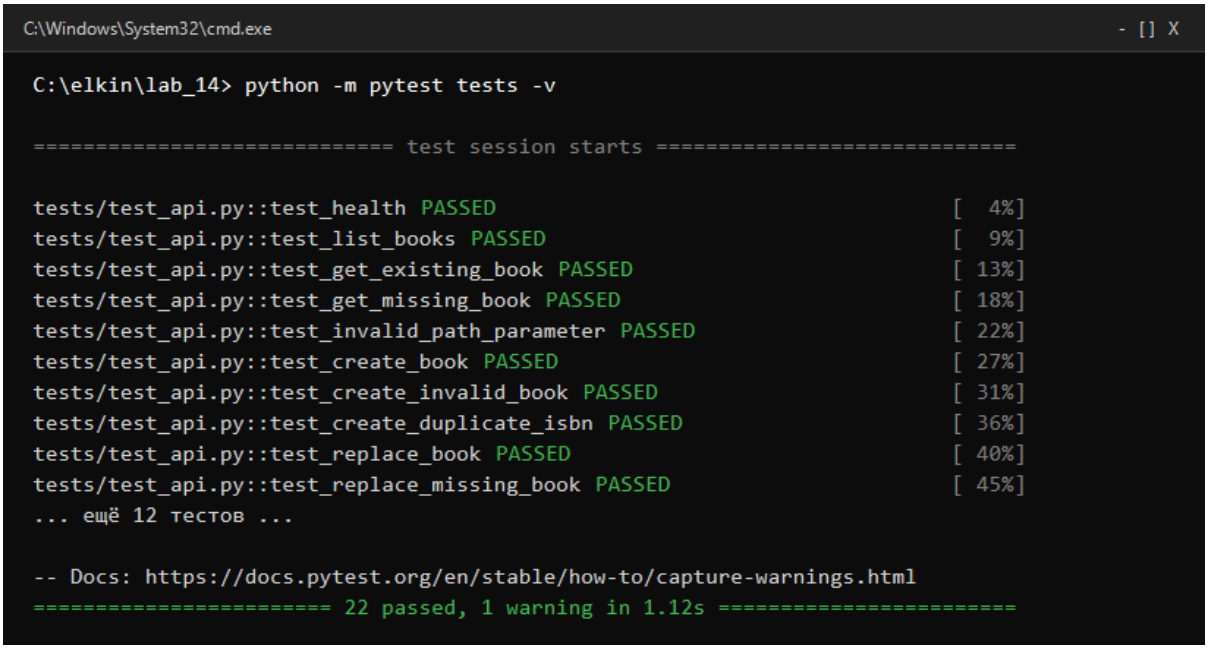


Рисунок 14.1 — Результат прохождения тестов REST API на FastAPI

Листинги ключевых фрагментов программы

Листинг 14.3 — Чтение с кэшированием (app/services/book.py)

```
def get_book(self, book_id: int) -> Dict[str, Any]:
    cached = self.cache.get(book_id)
    if cached is not None:
        return cached


    book = self.repository.get(book_id)
    if book is None:
        raise NotFoundError(book_id)


    payload = self._to_dict(book)
    self.cache.set(book_id, payload)
```

```
        return payload

    def delete_book(self, book_id: int) -> None:
        book = self.repository.get(book_id)
        if book is None:
            raise NotFoundError(book_id)
        self.repository.delete(book)
        self.cache.delete(book_id)
```

Листинг 14.4 — Создание приложения и обработка ошибок (app/main.py)

```
    def create_app(with_seed: bool = True) -> FastAPI:
        app = FastAPI(title="Library Books API", version="1.0.0")

        @app.middleware("http")
        async def log_requests(request: Request, call_next):
            started = time.perf_counter()
            response = await call_next(request)
            logger.info("%s %s -> %s (%.1f ms)", request.method,
                        request.url.path, response.status_code,
                        (time.perf_counter() - started) * 1000)
            return response

        @app.exception_handler(AppError)
        async def handle_app_error(request: Request, error: AppError):
            status_code = getattr(error, "status_code", 400)
            return JSONResponse(status_code=status_code,
                                content={"detail": str(error)})

        app.include_router(books.router)
        return app
```

Вывод

В ходе выполнения лабораторной работы разработано REST API библиотечного фонда на фреймворке FastAPI. Реализованы все основные операции над сущностью «книга», включая создание, чтение, полную и частичную замену, удаление, поиск, фильтрацию, сортировку, пагинацию и статистику. Схемы Pydantic обеспечили строгую валидацию входных данных и единый формат ответов, а коды состояния 200, 201, 204, 404, 409 и 422 — однозначную трактовку результатов. Разделение на маршруты, сервисы и репозитории, внедрение зависимостей и кэширование позволили получить расширяемое приложение, документация OpenAPI формируется автоматически, а двадцать два автоматических теста подтверждают корректность работы API.